

**UNIVERSITI TEKNOLOGI MALAYSIA
FACULTY OF COMPUTING**

TEST 2 (PRACTICAL)

SEMESTER I 2018/2019

SUBJECT CODE : SCSJ2013
SUBJECT NAME : DATA STRUCTURES AND ALGORITHMS
YEAR/COURSE : 2 (SCSJ/ SCSV/ SCSB/ SCSR/ SCSP)
TIME : 8.00 – 10.00 PM
DATE : 27 NOVEMBER 2018 (TUESDAY)
VENUE : N28 MPK1-MPK10

INSTRUCTIONS TO THE STUDENTS:

- This test consists of **TWO** questions. Answer all the questions.
- References to any resources by any means except **Data Structures and Algorithms Lab Module** are strictly prohibited.
- You are given **TWO HOURS** to complete the test inclusive the submission of your programs.

MATERIAL FOR THE TEST:

- You are provided a compressed file named **Test2_Resource.zip** containing two source code files, **Test2_Q1.cpp** and **Test2_Q2.cpp**, and the question, **Test2_ Question.pdf**.
- Download the file **Test2_Resource.zip** from the UTM's elearning and decompress it into the local hard drive of your computer.
- The provided program files should be used as the basis to answer this test.

SUBMISSION PROCEDURE:

- Compress all your source code files, i.e., **Test2_Q1.cpp** and **Test2_Q2.cpp** with the software WinRar into a single file named **Test2.rar** or **Test2.zip**.
- Submission must be done via the **UTM's e-learning system**.

Question 1

[30 Marks]

Given a C++ program file, **Test2_Q1.cpp**, which creates a linked list with the initial state as shown in Figure 1. The variables **head** and **tail** are pointers that pointing to the first and last node of the list, respectively.

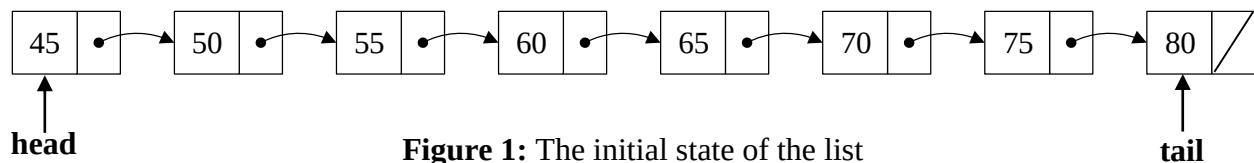


Figure 1: The initial state of the list

Append the appropriate lines of code at the space provided in **Test2_Q1.cpp** to accomplish each of the following tasks. Note that, each task is continuous from one to the next. The state of the list after the execution of each task is given in each question below. You must not change any line of code regarding the class definition and the given lines of code in the **main** function.

Task 1: Insert a new node (with the **item = 111**) at the end of the list. The result is given in Figure 2. (3 marks)

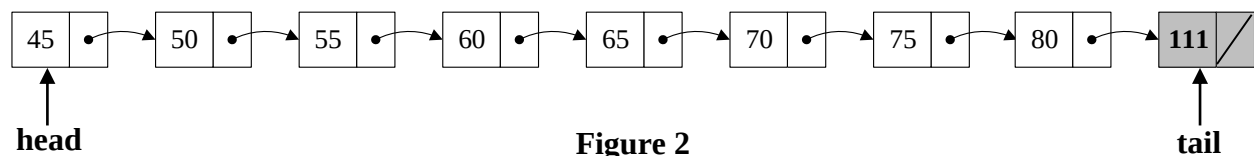


Figure 2

Task 2: Insert a new node (with the **item = 222**) between node 65 and node 70. The result is given in Figure 3. **Note:** In this task, you should use **loop**. (7 marks)

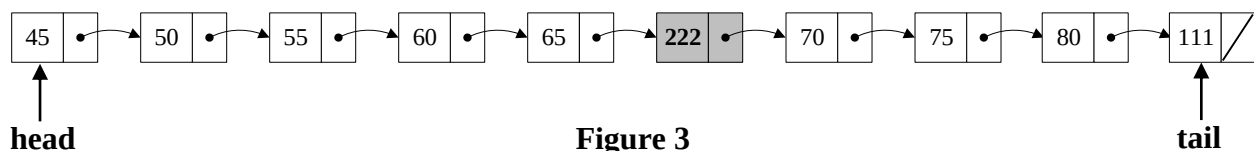


Figure 3

Task 3: Delete the first node from the list. Figure 4 shows the result of this task. (3 marks)

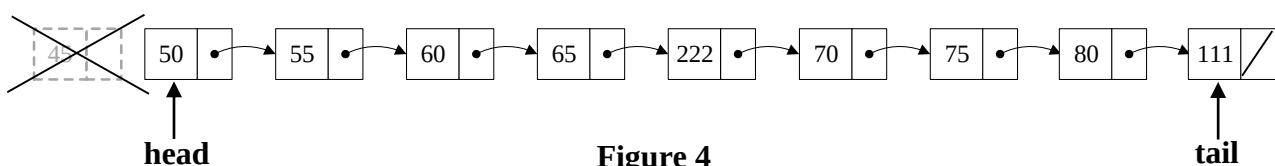
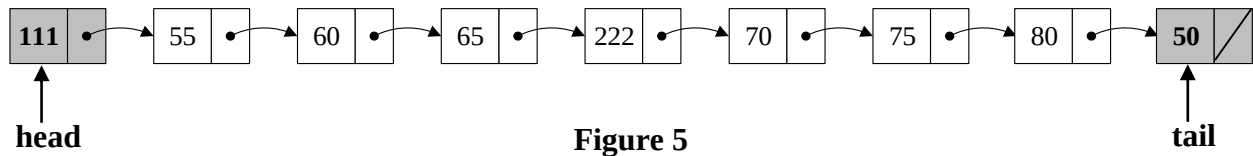


Figure 4

Task 4: Swap the positions of the first and last nodes in the list, i.e. the node with the **item = 111** becomes the first node, whereas the node with the **item = 50** becomes the last node. The result is given in Figure 5. **(9 marks)**



Task 5: Find a node in the list with the number that you have entered. Figure 6 shows the example of runs for this task. **Note:** value in **bold** is input by the user. **(8 marks)**

<pre>//Run 1 Enter a number that you want to find in the list: 90 90 is NOT in the list</pre>	<pre>//Run 2 Enter a number that you want to find in the list: 60 60 is in the list</pre>
--	--

Figure 6: Example of runs for Task 5

Question 2

[25 Marks]

Given a C++ program file, **Test2_Q2.cpp**, which contains a linked list implementation of stack to convert infix expression to postfix expression and to evaluate the postfix expression. The infix expression $(8 * (3 + 4) / 2 - 3 * 5)$ and the definition of the class **nodeStack** and **stack** are given in the program file. The variable **top** in the class **stack** is a pointer which points to the top node of the stack. The implementation of the method **createStack()**, **isEmpty()**, **stackTop()**, **push()** and **pop()** of the class **stack** are also given in the **Test2_Q2.cpp**. You must not change any line of code regarding the definition of the class **nodeStack** and **stack**. The method **isOperator()** is used to check if a character is an operator or not, whereas the method **precedence()** is used to determine the precedence of the operator.

Complete the implementation of **main** function by appending the appropriate lines of code at the space provided in **Test2_Q2.cpp** to accomplish each of the following tasks:

Conversion of infix expression to postfix expression using stack:

Task 1: If a left parentheses is encountered, push it into a stack. (2 marks)

Task 2: If an operator is encountered, then

- (a) Repeatedly pop from stack and add to postfix expression each operator on the top of stack, which has the same precedence as or higher precedence than the encountered operator.
- (b) Add the encountered operator to stack. (8 marks)

Task 3: If a right parentheses is encountered, then

- (a) Repeatedly pop from stack and add to postfix expression each operator on the top of stack until a left parentheses is encountered.
- (b) Remove the left parentheses. (5 marks)

Evaluation of postfix expression using stack:

Task 4: If an operator is encountered, then

- (a) Pop the first two operands in stack.

(b) Evaluate the encountered operator using both operands.

(c) Push the result of the evaluation to stack.

(10 marks)

Figure 7 and 8 show table of converting infix to postfix and table of evaluating postfix expression.

Figure 9 shows the expected output when the program runs.

Infix	Stack	Postfix
8*(3+4)/2-3*5	#	
*(3+4)/2-3*5	#	8
(3+4)/2-3*5	#*	8
3+4)/2-3*5	#*(	8
+4)/2-3*5	#*(	83
4)/2-3*5	#*(+	83
)2-3*5	#*(+	834
/2-3*5	#*	834+
2-3*5	#/	834+*
-3*5	#/	834+*2
3*5	#-	834+*2/
*5	#-	834+*2/3
5	#-*	834+*2/3
	#-*	834+*2/35
	#-	834+*2/35*
	#	834+*2/35*-

Figure 7: Table of converting infix to postfix

Postfix	Character	Operator	Operand 1	Operand 2	Result	Stack
834+*2/35*-	8					8
34+*2/35*-	3					83
4+*2/35*-	4					834
+*2/35*-	+	+	4	3	7	87
2/35-	*	*	7	8	56	56
2/35*-	2					562
/35*-	/	/	2	56	28	28
35*-	3					283
5*-	5					2835
*-	*	*	5	3	15	2815
-	-	-	15	28	13	13

Figure 8: Table of evaluating postfix expression

```
Convert infix to postfix
Infix: 8*(3+4)/2-3*5
Postfix: 834+*2/35*-

Evaluating postfix expression
Postfix expression: 834+*2/35*-
Result = 13
```

Figure 9: The output of the program